

Checkpoint 2 Report: Negative-Weight Single-Source Shortest Paths in Near-Linear Time

Team: Jibran Sheikh, Daniyal Farooqui

April 8, 2025

1 Problem and Contributions

1.1 Problem

The paper addresses the Single-Source Shortest Paths (SSSP) problem in directed graphs with negative edge weights. Traditional algorithms, such as Bellman-Ford, have a time complexity of $O(mn)$, which becomes impractical for large graphs. Recent approaches, such as those relying on min-cost flow reductions, introduce significant overhead due to their dependence on complex continuous optimization.

1.2 Main Contribution

The authors present a combinatorial algorithm that solves the negative-weight SSSP problem in $O(m \log^8 n \log W)$ time, where W is the absolute value of the most negative edge weight. This represents the first near-linear time solution for this problem, avoiding the need for complex optimization techniques and significantly simplifying prior methods.

2 Algorithmic Description

Input: A directed graph $G = (V, E)$ with integer edge weights (which may be negative), and a source vertex s .

Output: Shortest paths from s to all vertices, or detection of a negative-weight cycle.

2.1 Core Components

- **Price Functions:** Adjust edge weights to non-negative values using vertex potentials (as in Johnson's technique).
- **Low-Diameter Decomposition:**

- Partition the graph into strongly connected components (SCCs) with weak diameter D by removing a small set of edges E^{rem} .
- Ensures each SCC has $dist(u, v) \leq D$ for all $u, v \in \text{SCC}$.
- Probabilistic edge removal: $Pr[e \in E^{rem}] = O\left(\frac{w(e) \log^2 n}{D}\right)$.

- **Recursive Scaling (ScaleDown):**

- Recursively decompose the graph and eliminate negative weights.
- Uses **FixDAGEdges** to handle inter-component edges (DAG structure) and **ElimNeg** to resolve residual negative edges.

2.2 Detailed Explanation of Key Components

2.2.1 Price Functions

The **Price Functions** are used to reweight the edges of the graph so that the resulting weights become non-negative, making it possible to apply algorithms like Dijkstra's. This step is crucial for dealing with negative-weight edges efficiently. In this paper, the authors use a technique similar to *Johnson's framework*, where the edge weights are transformed using a potential function $\phi(v)$ assigned to each vertex v . The new weight of an edge (u, v) is computed as:

$$w'(u, v) = w(u, v) + \phi(u) - \phi(v)$$

This ensures that all edge weights are non-negative, as the transformation adjusts the original weight by the difference between the potentials of the vertices. The potential function $\phi(v)$ is typically computed using a modified version of the *Bellman-Ford* algorithm, ensuring that the reweighted graph preserves the shortest path structure of the original graph. By converting all edge weights to non-negative values, the problem becomes solvable by standard shortest path algorithms, such as Dijkstra's, in linear time.

2.2.2 Low-Diameter Decomposition (LDD)

The **Low-Diameter Decomposition (LDD)** is a technique used to simplify the graph by dividing it into smaller subgraphs, making the overall shortest path problem easier to handle. The LDD algorithm partitions the graph into *strongly connected components (SCCs)*, where each component has a **weak diameter* of at most D . The weak diameter of a component is defined as the maximum shortest path between any two vertices within that component.

To achieve this, a probabilistic edge removal technique is used. A set of edges, E^{rem} , is removed such that the remaining graph is decomposed into SCCs with the desired weak diameter. The removal probability for each edge is computed as:

$$Pr[e \in E^{rem}] = O\left(\frac{w(e) \log^2 n}{D}\right)$$

where $w(e)$ is the weight of the edge, n is the number of vertices, and D is the bound on the diameter of the SCCs. This random edge removal ensures that the resulting graph has smaller, manageable subgraphs with guaranteed diameter bounds, which reduces the complexity of finding the shortest paths. Each SCC is then processed individually, and their interconnections are handled using the recursive scaling technique. By breaking the graph into smaller components with bounded diameters, the problem becomes easier to solve, as the maximum path length within each component is constrained.

2.3 High-Level Workflow

1. Scaling Framework:

- Scale edge weights to ensure all weights are ≥ -1 .
- Use `SPmain` to iteratively adjust prices via recursive calls to `ScaleDown`.

2. Key Subroutines:

- **ElimNeg**: Hybrid of Dijkstra and Bellman-Ford to handle graphs with a few negative edges.
- **FixDAGEdges**: Ensures non-negative weights across DAG components.

2.4 Example

For a graph G with negative edges:

- Apply `LowDiamDecomposition` to split G into SCCs.
- Recursively fix weights inside SCCs using `ScaleDown`.
- Adjust inter-component edges via `FixDAGEdges`.
- Finally, run `ElimNeg` to resolve remaining negative edges.

3 Comparison with Existing Approaches

Method	Time Complexity	Key Limitations	Novelty of This Work
Bellman-Ford	$O(mn)$	Too slow for large graphs.	Near-linear time via combinatorial techniques.
Dijkstra's Algorithm	$O(m + n \log n)$	Requires non-negative weights.	Generalizes to negative-weight graphs with combinatorial techniques.
Min-Cost Flow Reductions	$O(m^{4/3} \log W)$	Requires complex continuous optimization.	Purely combinatorial, simpler implementation.

Table 1: Comparison of approaches for negative-weight SSSP.

4 Data Structures and Techniques

- **Graph Decomposition:** Low-diameter partitioning via randomized edge removal.
- **Priority Queues:** Used in `ElimNeg` for Dijkstra-like phases.
- **Dynamic Programming:** For `FixDAGEdges` to assign prices in topological order.
- **Recursive Scaling:** Reduces edge weight magnitudes iteratively.
- **Probabilistic Analysis:** Bounding edge removal probabilities in decomposition.

5 Implementation Outlook

5.1 Challenges

- **Low-Diameter Decomposition:**
 - Efficiently computing SCCs with diameter guarantees.
 - Handling probabilistic edge removal while maintaining correctness.
- **Recursive Scaling:**
 - Managing multiple scaling phases and ensuring termination.
 - Numerical precision with large W , mitigated by integer weights.
- **Hybrid Algorithms:**
 - Integrating Dijkstra and Bellman-Ford in `ElimNeg` without priority queue bottlenecks.

5.2 Strategies

- **Prototyping:** Implement core subroutines (e.g., `ScaleDown`, `LowDiamDecomposition`) in Python/NetworkX for validation.
- **Optimization:** Switch to C++/Boost Graph for performance-critical components (e.g., priority queues).
- **Testing:** Use synthetic graphs with controlled negative cycles for correctness checks.